



Artificial Intelligence

By Jawad Jammoul

Content:

Chapter 01 : Introduction to Artificial Intelligence	page
	01
Chapter 02 : Intelligent Agents	02
Chapter 03 : Solving problems by searching	07
Chapter 04 : Adversial Search	11
Chapter 05 : First-Order Logic	14
Chapter 06 : Inference in First-Order Logic	27

Discrete Mathematics:

Chapter 01 : The Foundations - Logic and Proofs[1]	37
Chapter 02 : The Foundations - Logic and Proofs[2]	41

Chapter 01 : Introduction to Artificial Intelligence

01*What is AI ?

AI is the ability of machines or computer systems to perform tasks that normally require human intelligence , such as learning , reasoning , and decision-making

02*Birth of AI:

A-Classical Period (1950s–1970s):

This is the early stage of AI when researchers tried to make machines follow logical rules and solve problems using strict step-by-step instructions (symbolic AI).

Example: Early programs that solved math problems or proved simple theorems using predefined rules.

B-Romantic Period (1970s–1990s):

During this period, AI research slowed down and expectations became more realistic. Scientists focused on knowledge-based systems and trying to imitate human expertise rather than full intelligence.

Example: Expert systems used in medicine to help doctors diagnose diseases based on stored rules.

C-Modern Period (1990s–present):

AI became powerful due to big data, faster computers, and machine learning techniques that allow systems to learn from data instead of fixed rules.

Example: AI systems used in medical diagnosis, like detecting cancer from MRI scans using deep learning models.

03*Acting Humanly : The Turing Test approach

Means building AI systems that behave in a way that is indistinguishable from human behavior. The goal is not just to solve problems correctly , but to respond communicate , and act like a real person in different situations

The Turing Test , proposed by Alan Turing , is a method to evaluate whether a machine can exhibit human-like intelligence. In this test, if a human cannot reliably distinguish between responses from a machine and a real person during a conversation , the machine is considered intelligent

04*Thinking Humanly : The cognitive modeling approach

Means designing AI systems that try to understand and reproduce how the human mind works. The focus is not only on solving problems, but on simulating human thought processes such as memory, reasoning, learning, and decision-making.

The cognitive modeling approach studies how humans think by observing psychology and brain behavior, then builds AI systems that follow similar mental processes. The goal is to make the computer “think” in a way similar to humans, not just act correctly.

05*Thinking Rationally : The “Laws of thought” approach

Means designing AI systems that make decisions based on logic and correct reasoning rules. The goal is to ensure that the machine reaches a logically valid conclusion, even if it does not think like a human.

The “Laws of thought” approach is based on formal logic principles that define correct reasoning. AI systems using this approach try to represent knowledge in logical statements and use inference rules to derive conclusions that are always logically valid.

06*Acting Rationally : The rational agent approach

Means designing AI systems that choose actions that maximize the chance of achieving the best possible outcome. The system does not need to think like humans, but it must act in a way that is most efficient and beneficial based on the available information.

The rational agent approach defines AI as an “agent” that perceives its environment through sensors and acts on it through actuators, always aiming to achieve the best expected result. It evaluates different actions and chooses the one that maximizes performance or success.

Example: A self-driving car that constantly analyzes road conditions, traffic, and obstacles, then decides the safest and most efficient driving actions in real time.

Chapter 02 : Intelligent Agents

01*What is an Agent?

An **agent** is anything that can perceive its environment through sensors and act upon that environment using actuators to achieve a goal. It takes input from the world, processes it, and produces an output action.

02*Percepts and actions:

Percepts are the inputs that an agent receives from its environment through sensors. They represent what the agent perceives or observes at a specific moment or over time.

Actions are the outputs or operations that an agent performs on its environment through actuators based on its percepts.

03*Agent Program & Agent Function:

The **agent function** is a mathematical description that maps every possible sequence of percepts (what the agent has observed so far) to an action. It defines what the agent should do in every possible situation.

The **agent program** is the actual implementation (code) that runs on a computer to realize the agent function. It takes percepts as input and produces actions as output based on programmed rules or learning methods.

04*Vacuum cleaner agent:

A **vacuum cleaner** agent is a simple AI example that operates in an environment (like a room or grid) and tries to clean dirt based on what it perceives. It uses sensors to detect whether the current location is dirty or clean, and then performs actions like moving or sucking dirt.

05*Rational Agent:

A **rational agent** is an agent that always chooses the action that is expected to achieve the best outcome or maximize its performance measure, given the percepts it has. It does not need to be perfect, but it must act in the most efficient way based on available information.

06*Performance measure:

A **performance measure** is a clear criterion used to evaluate how successful an agent is at achieving its goal in an environment. It defines what “good behavior” means for the agent and is used to judge whether the agent is acting rationally.

07*Rationality:

Rationality is the property of an agent that chooses actions which maximize its expected performance measure based on the percepts it has received and the knowledge it has. A rational agent does the best it can with the information available at the time, even if the outcome is not always perfect.

08*Omniscience, Learning and autonomy:

Omniscience means having complete knowledge about everything and always knowing the correct outcome of every action. In AI, it is impossible in real-world environments because the agent cannot know all future events or hidden information.

Learning is the ability of an agent to improve its performance over time by gaining experience from percepts and feedback. Instead of being fully pre-programmed, a learning agent adapts to new situations.

Autonomy refers to how independently an agent can operate without relying heavily on built-in knowledge from humans. A highly autonomous agent learns from its own experience and makes decisions on its own.

09*Task Environment:

A **task environment** is the external world in which an AI agent operates. It includes everything the agent interacts with, such as inputs, outputs, rules, and conditions that define the problem it must solve.

Specifying the Task Environment:

To design an AI agent properly, we must clearly define its task environment: what the agent observes, what actions it can take, and what goals it must achieve. This helps us understand the problem the agent is solving.

PEAS:

PEAS is a framework used to describe an AI agent's task environment

Performance measure: How success is evaluated

Environment: The world in which the agent operates

Actuators: How the agent acts on the environment

Sensors: How the agent perceives the environment

Properties of Task Environments:

Task environments can have different properties

Fully observable vs partially observable: Does the agent see everything or only part of the environment?

Deterministic vs stochastic: Are outcomes predictable or random?

Episodic vs sequential: Does each action depend on previous actions?

Static vs dynamic: Does the environment change while the agent is thinking?

Discrete vs continuous: Are states/actions limited or continuous?

Single-agent vs multi-agent: Is there only one agent or many interacting agents?

10*The Structure of Agents:

The **agent program** is the software or set of instructions that determines what action the agent should take based on its percepts. It implements the agent function, which maps percept sequences to actions.

Architecture is the physical or hardware platform on which the agent program runs. It provides the sensors and actuators needed for interaction with the environment.

An intelligent agent is created by combining the architecture with the agent program:

AI Agent = Architecture + Agent Program

11*Types of Agent Programs:

1-Simple Reflex Agent:

A **simple reflex agent** acts only based on the current percept, using condition–action rules without considering past information.

- React only to the current percept
- Use condition–action rules
- Do not store past information
- Suitable for simple environments

2-Model-Based Reflex Agent:

A **model-based reflex agent** keeps an internal model of the environment to track aspects that cannot be directly observed. It uses both current percepts and stored information to make decisions.

- Maintain an internal model of the environment
- Use past percepts with current information
- Can handle partially observable environments
- Track changes in the environment over time

3-Goal-Based Agent:

A **goal-based agent** chooses actions by considering future consequences and selecting actions that help achieve a specific goal.

- Act to achieve specific goals
- Consider future consequences of actions
- Compare different possible actions
- More flexible than reflex agents

4-Utility-Based Agent:

A **utility-based agent** selects actions that maximize overall satisfaction or “utility,” especially when several goals or choices are possible.

- Maximize utility or overall satisfaction
- Evaluate multiple possible outcomes
- Handle trade-offs between different goals
- Choose the most beneficial action

12*Learning Agents:

A learning agent is an AI agent that can improve its performance over time by learning from experience and feedback from the environment. Instead of relying only on fixed rules, it adapts its behavior to perform tasks more effectively.

Main Components of a Learning Agent:

- **Learning element:** Improves the agent’s knowledge and behavior
- **Performance element:** Chooses external actions
- **Critic:** Evaluates the agent’s performance and provides feedback
- **Problem generator:** Suggests new actions for learning and exploration

13*Advantages of Simpler Environments:

- Easier to understand and model
- Require less computational power
- Faster decision-making and learning
- Fewer errors due to reduced complexity
- Easier testing and debugging

Chapter 03 : Solving problems by searching

01*Problem Solving Agents:

A problem-solving agent is an AI agent that decides what to do by formulating a problem and then searching for a sequence of actions that leads to a goal state. It uses a systematic approach to move from an initial state to a desired goal state.

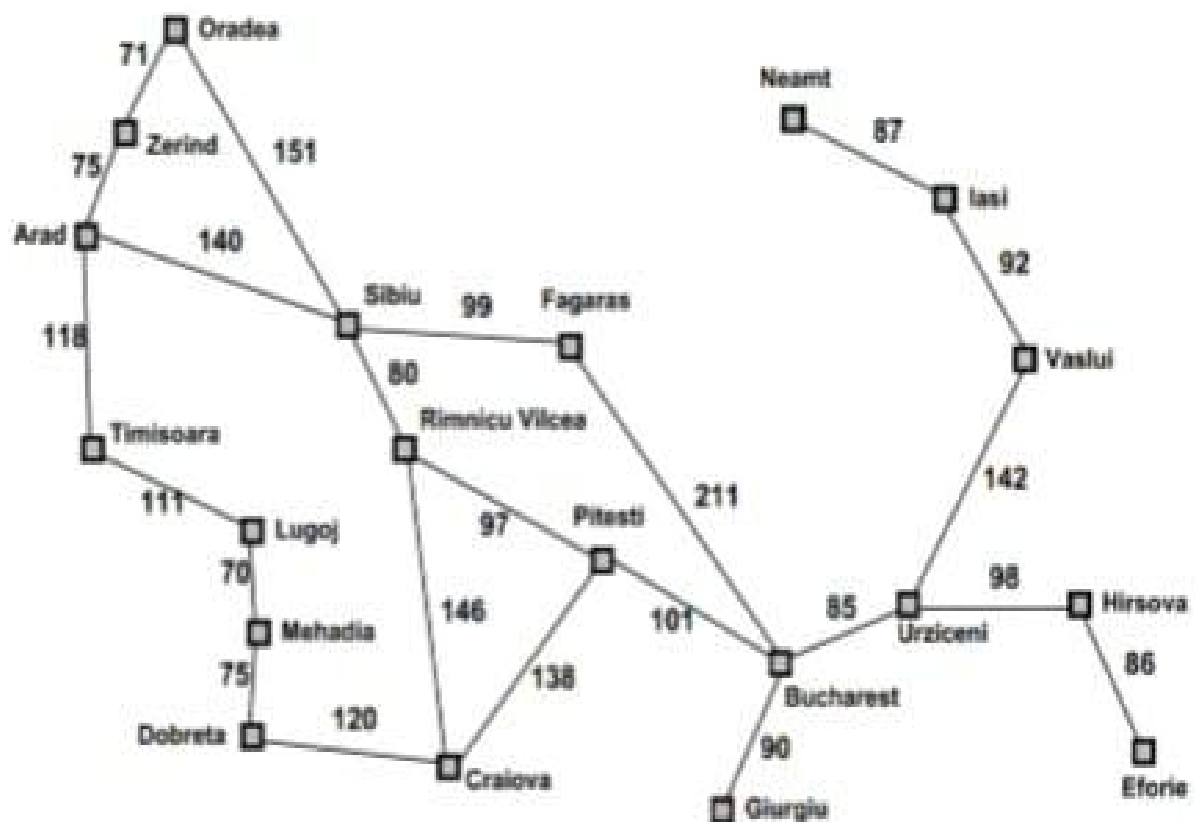
Goal formulation: Defines what it wants to achieve

Problem formulation: Defines states, actions, and the environment

Search: Finds a sequence of actions (solution)

Execution: Performs the actions to reach the goal

Map of Romania:



02*Search Problem Components:

1-Initial State : The starting point of the problem, where the agent begins its search.

2-State Space : The set of all possible states that can be reached from the initial state by applying actions.

3-Actions (Operators) : The possible moves the agent can take from any state.

4-Transition Model (Result Function) : Defines what happens when an action is applied to a state, i.e., it returns the next state.

5-Goal Test : A function that checks whether a given state is the goal state.

6-Path Cost : A numerical value that measures the cost of a path (time, distance, steps, etc.).

03*Formulating Problems:

Abstraction means simplifying the real world by removing unnecessary details and keeping only the information that is relevant for solving the problem. This helps reduce complexity and makes the search process easier and faster.

Why Abstraction is Important?

- Reduces complexity of the problem
- Makes search faster and more efficient
- Focuses only on relevant information
- Helps represent real-world problems in a simplified model

04*State Space Graphs:

A state space graph is a representation of all possible states of a problem and the transitions (actions) between them. It shows how an AI agent can move from the initial state to the goal state.

A node represents a specific state in the problem space. Each node corresponds to a possible configuration of the problem at a certain point.

The goal test is a function applied to a node to check whether it is the goal state or not. It returns true if the node satisfies the goal condition.

05*Search tree:

A **search tree** is a tree representation used in AI search problems to explore possible paths from the initial state to the goal state. It is built during the search process and shows how different actions lead to new states.

Searching for solution:

Leaf Node : A node with no children (no further expansion).

Frontier : All generated but not yet expanded nodes.

Search Strategy : A method used to choose which node to expand next.

Loopy Path : A path that repeats the same state and forms a cycle.

06*Graph Search:

Graph search is a search method that explores a state space while avoiding repeated states by keeping track of explored nodes (closed list) to prevent loops and redundant work.

07*Infrastructure for search algorithms:

Refers to the basic components needed to implement and run any search method in AI.

08*Measuring problem-solving performance:

Problem-solving performance is measured by evaluating how well a search algorithm finds solutions efficiently and correctly.

Completeness : Whether the algorithm always finds a solution if one exists.

Optimality : Whether the algorithm finds the best (lowest cost) solution.

Time Complexity : How much time the algorithm takes to find a solution.

Space Complexity : How much memory the algorithm needs during the search process.

09*Uninformed Search Strategies:

Uninformed search strategies are search methods that do not use any additional knowledge about the goal. They only use the information provided in the problem definition.

1-Breadth-First Search (BFS):

Expands the shallowest nodes first level by level.

2-Depth-First Search (DFS):

Expands the deepest node first and goes as far as possible before backtracking.

3-Uniform-Cost Search (UCS):

Expands the node with the lowest path cost first.

4-Depth-Limited Search (DLS):

DFS with a fixed depth limit to avoid infinite paths.

5-Iterative Deepening Search (IDS):

Repeatedly applies DFS with increasing depth limits.

Uninformed Search (Blind Search)	Informed Search (Heuristic Search)
Uninformed search uses only the information given in the problem definition and does not use any extra knowledge about the goal. Example: BFS, DFS, UCS.	Informed search uses additional knowledge (heuristics) to estimate how close a state is to the goal and guide the search more efficiently. Example: A* search, Greedy best-first search.

10*Comparing Uninformed Search Strategies:

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative-Deepening
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a
Time	$O(b^d)$	$O(b^{1+L \lceil C*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+L \lceil C*/\epsilon \rceil})$	$O(bm)$	$O(b^l)$	$O(bd)$
Optimal?	Yes ^c	Yes	No	No	Yes ^c

11*Informed Search-Heuristic Function:

$h(n)$ =estimated cost of the cheapest path from the state at node n to a goal state

12*Greedy best-first search:

Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function; that is, $f(n)=h(n)$.

13*A* Search:

The most widely known form of best-first search is called A* search (pronounced "A-star search"). It evaluates nodes by combining $g(n)$, the cost to reach the node, and $h(n)$, the cost to get from the node to the goal:

$$f(n)=g(n)+h(n).$$

14*Optimality of A*:

Because $g(n)$ is the actual cost to reach n along the current path, and $f(n)=g(n)+h(n)$, we have as an immediate consequence that $f(n)$ never overestimates the true cost of a solution along the current path through n .

A heuristic $h(n)$ is consistent if, for every node n and every successor n' of n generated by any action a , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n' :

$$h(n) \leq c(n,a,n') + h(n')$$

The proof follows directly from the definition of consistency. Suppose n' is a successor of n , then:

$$g(n') = g(n) + c(n,a,n') \text{ for some action } a,$$

$$\text{and we have } f(n') = g(n') + h(n') = g(n) + c(n,a,n') + h(n') \geq g(n) + h(n) = f(n).$$

Chapter 04 : Adversarial Search

01*Games definitions:

S0: The initial state, which specifies how the game is set up at the start.

PLAYER(s): Defines which player has the move in a state.

ACTIONS(s): Returns the set of legal moves in a state.

RESULT(s, a): The transition model, which defines the result of a move.

TERMINAL-TEST(s): A terminal test, which is true when the game is over and false otherwise.

States where the game has ended are called terminal states.

UTILITY(s, p):

A **utility function** (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p .

A **zero-sum game** is (confusingly) defined as one where the total payoff to all players is the same for every instance of the game.

The initial state, ACTIONS function, and RESULT function define the game tree for the game, a tree where the nodes are game-states and the edges are moves.

02*Optimal Decisions in Games:(Minimax strategy)

MAX must find a strategy, which specifies MAX's move in the initial state, then MAX's moves in the states resulting from MIN, then MAX's moves in the states resulting from every possible response by MIN to those moves, and so on.

An optimal strategy leads to outcomes at least as good as any other strategy when one is playing an infallible opponent

Each half-moves (MAX move or MIN move) is called a ply.

Given a game tree, the optimal strategy can be determined from the minimax value of each node, which we write as MINIMAX(n).

MINIMAX(s) =

UTILITY(s) if TERMINAL-TEST(s)

max $a \in \text{Actions}(s)$ MINIMAX(RESULT(s, a)) if PLAYER(s) = MAX

min $a \in \text{Actions}(s)$ MINIMAX(RESULT(s, a)) if PLAYER(s) = MIN

03*The minimax Algorithm characteristics:

-The **minimax algorithm** characteristics:

-The minimax algorithm computes the minimax decision from the current state.

-The minimax algorithm uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.

-The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.

-In the Trivial Game tree, the algorithm first recurses down to the three bottom left nodes and uses the UTILITY function on them to discover that their values are 3, 12, and 8, respectively. Then it takes the minimum of these values, 3, and returns it as the backed-up value of node B.

-A similar process gives the backed-up values of 2 for C and 2 for D. Finally, we take the maximum of 3, 2, and 2 to get the backed-up value of 3 for the root node.

-The minimax algorithm Performance:

-The minimax algorithm performs a complete depth-first exploration of the game tree.

-If the maximum depth of the tree is m and there are b legal moves at each point, then:

The time complexity of the minimax algorithm is $O(bm)$.

The space complexity is $O(bm)$ for an algorithm that generates all actions at once, or $O(m)$ for an algorithm that generates actions one at a time.

-For real games, of course, the time cost is totally impractical, but this algorithm serves as the basis for the mathematical analysis of games and for more practical algorithms.

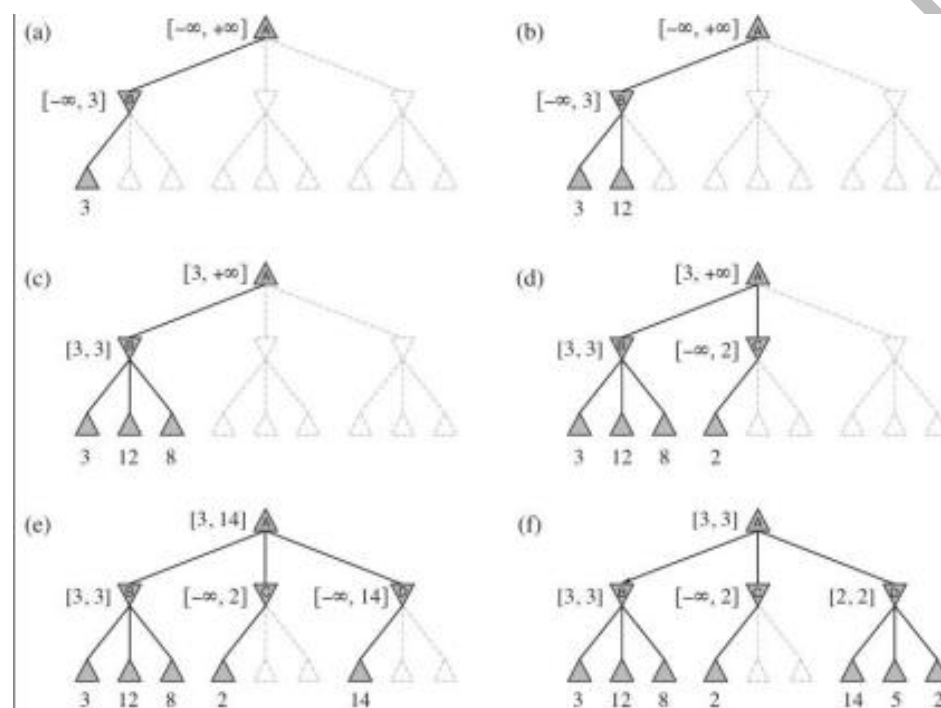
04*Alpha-beta pruning:

- The problem with minimax search is that the number of game states it must examine is exponential in the depth of the tree.
- Unfortunately, we can't eliminate the exponent, but it turns out we can effectively cut it in half.
- The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree.

Another way to look at this is as a simplification of the formula for MINIMAX.

Let the two un-evaluated successors of node C have values x and y. Then the value of the root node is given by

$$\begin{aligned} \text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) = \max(3, z, 2) \text{ where } z = \min(2, x, y) \leq 2 \\ &= 3. \end{aligned}$$



On a MAX node:

First, assign $v = -\infty$

We check if $v \geq \beta$ then we return v

We assign $\alpha \leftarrow \max(\alpha, v)$

On a MIN node:

First, we assign $v = +\infty$

We check if $v \leq \alpha$ then we return v

We assign $\beta \leftarrow \min(\beta, v)$

Chapter 05 : First-Order Logic

01*Representation Revisited:

Our discussion motivates the development of first-order logic.

-Programs written in programming languages such as Java or C++ are only computational processes.

-Programming languages variables and data structures can represent facts but lack any general mechanism

for deriving facts from other facts.

-In a programming language we can use a 4×4 array to represent the contents of the wumpus world:

$\text{World}[2,2] \leftarrow \text{Pit}$ is a natural way to assert that there is a pit in square $[2,2]$.

Cannot represent: There is a pit in $[2,2]$ or $[3,1]$

Cannot represent: If the Wumpus is in $[1,1]$ then he is not in $[2,2]$

For example, we were forced to write a separate rule about breezes and pits for each square, such as: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ "Squares adjacent to pits are breezy."

-The language of thought:

Natural languages (English, Arabic ...) are very expressive, but cannot be used for reasoning systems since it has a lot of rules.

Natural language sentences may reflect multiple meaning to our minds depending on the context. To store a sentence into a knowledge base, we need to store the context, here a new problem arise: how to store context?

Natural language sentences are ambiguous, which means that the same sentence may reflect different thoughts into our minds.

The famous Sapir–Whorf hypothesis claims that our understanding of the world is strongly influenced by the language we speak. The French have two words “chaise” and “fauteuil,” for a concept that English speakers cover with one: “chair.” But English speakers can easily recognize the category fauteuil and give it a name, roughly “open-arm chair”.

Wanner (1974) did an experiment and found that subjects made the right choice at chance level, about 50% of the time, but remembered the content of what they read with better than 90% accuracy. This suggests that people process the words to form some kind of nonverbal representation. Can you remember which of the following two phrases formed the opening of Section 8.1:
“In this section, we discuss the nature of representation languages ...”
“This section covers the topic of knowledge representation languages ...”

Language also seems to influence thought through seemingly arbitrary grammatical features such as the gender of nouns:

For example, “bridge” is masculine in Spanish and feminine in German. Boroditsky (2003) asked subjects to choose English adjectives to describe a photograph of a particular bridge. Spanish speakers chose big, dangerous, strong, and towering, whereas German speakers chose beautiful, elegant, fragile and slender.

Loftus and Palmer (1974) showed experimental subjects a movie of an auto accident. Subjects who were asked “How fast were the cars going when they contacted each other?” reported an average of 32 mph, while subjects who were asked the question with the word “smashed” instead of “contacted” reported 41mph for the same cars in the same movie.

In a first-order logic reasoning system that uses CNF (Conjunctive Normal Form), we can see that the linguistic form “ $\neg(A \vee B)$ ” and “ $\neg A \wedge \neg B$ ” are the same because we can look inside the system and see that the two sentences are stored as the same canonical CNF form. Can we do that with the human brain? Until recently the answer was “no”, now it is maybe.

Combining the best of formal and natural languages :

We can adopt the foundation of propositional logic, a declarative, compositional semantics that is context-independent and unambiguous and build a more expressive logic on that foundation, borrowing representational ideas from natural language while avoiding its drawbacks.

When we look at the syntax of natural language, the most obvious elements are nouns and noun phrases that refer to objects (squares, pits, wumpuses) and verbs and verb phrases that refer to relations among objects (is breezy, is adjacent to, shoots). Some of these relations are functions, relations in which there is only one "value" for a given "input."

It is easy to start listing examples of objects, relations, and functions:

Objects: people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries ...

Relations:

these can be unary relations or properties such as red, round, bogus, prime, multistoried ..., or more general n-ary relations such as brother of, bigger than, inside, part of, has color, occurred after, owns, comes between, ...

Functions:

father of, best friend, third inning of, one more than, beginning of ...

First Order Logic Ontological Commitment: Facts, Objects and Relations

First Order Logic Epistemological Commitment: true/false/unknown

02*Syntax and Semantics of first-order-logic:

-First-Order Logic-Model:

Models of a logical language are the formal structures that constitute the possible worlds under consideration.

-Each model links the vocabulary of the logical sentences to elements of the possible world, so that the truth of any sentence can be determined.

-Models for first-order logic have objects in them:

The domain of a model is the set of objects or domain elements it contains.

The domain is required to be non-empty.

Every possible world must contain at least one object.

-Objects:

Richard the Lionheart, King of England from 1189 to 1199;

His younger brother, the evil King John, who ruled from 1199 to 1215;

The left leg of Richard

The left leg of John

A crown.

Relations:

Brotherhood Relation:

{<Richard the Lionheart, King John>, <King John, Richard the Lionheart> }

On Head Relation:

{<the crown, King John>}

The “brotherhood” and “on head” relations are binary relations, that is, they relate pairs of objects.

Unary Relations or Properties:

Person: {<Richard>, <John>}

King: {<John>}

Crown: {<crown>}

Functions:

Left Leg: Richard the Lionheart → Richard’s left leg

King John → John’s left leg

-First-Order Logic-Symbols and interpretations

Symbols:

Constant Symbols: which stand for objects.

Predicate Symbols: which stand for relations.

Function Symbols: which stand for functions.

We adopt the convention that these symbols will begin with uppercase letters.

Each predicate and function symbol comes with an arity that fixes the number of arguments

Interpretation (Intended Interpretation):

An interpretation maps constant symbols to objects, predicate symbols to relations on those objects, and function symbols to functions on those objects.

Sample Model Interpretation:

Richard refers to Richard the Lionheart and John refers to the evil King John.

Brother refers to the brotherhood relation.

OnHead refers to the “on head” relation that holds between the crown and King John.

Person, King, and Crown refer to the sets of objects that are persons, kings, and crowns.

LeftLeg refers to the “left leg” function.

Notice that not all the objects need have a name, for example, the intended interpretation does not name the crown or the legs.

-First-Order Logic - Terms:

A term is a logical expression that refers to an object.

Constant symbols are therefore terms, but it is not always convenient to have a distinct symbol to name every object.

A complex term is formed by a function symbol followed by a parenthesized list of terms as arguments to the function symbol.

It is important to remember that a complex term is just a complicated kind of name.

It is not a “subroutine call” that “returns a value.”

Consider a term $f(t_1, \dots, t_n)$:

The function symbol f refers to some function in the model (call it F)

The argument terms refer to objects in the domain (call them $d_1, \dots, d_n$).

The term refers to the object that is the value of the function F applied to $d_1, \dots, d_n$.

For example, $\text{LeftLeg}(\text{John})$ refers to King John’s left leg.

-First-Order Logic - Sentences:

Atomic Sentence: is formed from a predicate symbol optionally followed by a parenthesized list of terms, such as:

$\text{Brother}(\text{Richard}, \text{John})$

This states, under the intended interpretation given earlier, that Richard the Lion heart is the brother of King John.

Atomic sentences can have complex terms as arguments. Thus:

$\text{Married}(\text{Father}(\text{Richard}), \text{Mother}(\text{John}))$

States that Richard the Lionheart’s father is married to King John’s mother (again, under a suitable interpretation).

An atomic sentence is true in each model if the relation referred to by the predicate symbol holds among the objects referred to by the arguments.

Complex Sentence: is constructed by applying logical connectives to construct more complex sentences, with

the same syntax and semantics as in propositional logic. Here are four sentences that are true in the sample

model under our intended interpretation:

$\neg \text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John})$

$\text{Brother}(\text{Richard}, \text{John}) \wedge \text{Brother}(\text{John}, \text{Richard})$

$\text{King}(\text{Richard}) \vee \text{King}(\text{John})$

$\neg \text{King}(\text{Richard}) \Rightarrow \text{King}(\text{John})$.

-First-Order Logic - Quantifiers:

Once we have a logic that allows objects, it is natural to express properties of entire

collections of objects, instead of enumerating the objects by name.

First order logic has two types of quantifiers:

Universal Quantification ($\forall$)

Existential quantification ($\exists$)

-Universal Quantification $\forall$:

$\forall$ is usually pronounced "For all ...".

Intuitively, the sentence $\forall xP$, where P is any logical expression, says that P is true for every object x . Where x is any possible object in the model.

By asserting the universally quantified sentence, which is equivalent to asserting a whole list of individual implications, we end up asserting the conclusion of the rule just for those objects for whom the premise (Left Side of the $\Rightarrow$) is true and saying nothing at all about those individuals for whom the premise is false.

"All kings are persons" is written in first-order logic as:

$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$.

Thus, the sentence says, "For all x , if x is a king, then x is a person."

The symbol x is called a variable. By convention, variables are lowercase letters. A variable is a term all by itself, and as such can also serve as the argument of a function, for example, $\text{LeftLeg}(x)$. A term with no variables is called a ground term.

This equivalent to the following assertions:

Richard the Lionheart is a king $\Rightarrow$ Richard the Lionheart is a person

King John is a king $\Rightarrow$ King John is a person

Richard's left leg is a king $\Rightarrow$ Richard's left leg is a person

John's left leg is a king $\Rightarrow$ John's left leg is a person

The crown is a king $\Rightarrow$ the crown is a person.

A common mistake, made frequently even by diligent readers who have read this paragraph

several times, is to use conjunction instead of implication.

The sentence:

$\forall x \text{ King}(x) \wedge \text{Person}(x)$ would be equivalent to asserting:

Richard the Lionheart is a king $\wedge$ Richard the Lionheart is a person

King John is a king $\wedge$ King John is a person

Richard's left leg is a king $\wedge$ Richard's left leg is a person and so on.

Obviously, this does not capture what we want.

Nested Quantification:

We will often want to express more complex sentences using multiple quantifiers. The simplest case is where the

quantifiers are of the same type. For example, “Brothers are siblings” can be written as:

$$\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

Consecutive quantifiers of the same type can be written as one quantifier with several variables. For example, to say

that siblinghood is a symmetric relationship, we can write:

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x)$$

In other cases, we will have mixtures. “Everybody loves somebody” means that for every person, there is someone that

person loves: $\forall x \exists y \text{ Loves}(x, y)$

On the other hand, to say “There is someone who is loved by everyone,” we write: $\exists y \forall x \text{ Loves}(x, y)$

The order of quantification is therefore very important. It becomes clearer if we insert parentheses. $\forall x (\exists y \text{ Loves}(x, y))$

says that everyone has a particular property, namely, the property that they love someone.

On the other hand, $\exists y$

$(\forall x \text{ Loves}(x, y))$ says that someone in the world has a particular property, namely the property of being loved by everybody.

Some confusion can arise when two quantifiers are used with the same variable name.

Consider the sentence $\forall x (\text{Crown}(x) \vee (\exists x \text{ Brother}(\text{Richard}, x)))$. To avoid the confusion between x within $\text{Crown}(x)$ and x within $\text{Brother}(\text{Richard}, x)$, it is better to use another variable: $\forall x (\text{Crown}(x) \vee (\exists z \text{ Brother}(\text{Richard}, z)))$.

The rule is that the variable belongs to the innermost quantifier that mentions it, then it will not be subject to any other quantification.

-Connection between $\forall$ and $\exists$:

$$\forall_x \neg P \equiv \neg \exists_x P \quad \neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

$$\neg \forall_x P \equiv \exists_x \neg P \quad \neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\forall_x P \equiv \neg \exists_x \neg P \quad P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$

$$\exists_x P \equiv \neg \forall_x \neg P \quad P \vee Q \equiv \neg(\neg P \wedge \neg Q).$$

De Morgan Rules

The two quantifiers are intimately connected with each other, through negation.

Asserting that everyone dislikes parsnips is the same as asserting there does not exist someone who likes them, and vice versa:

$\forall x \neg \text{Likes}(x, \text{Parsnips})$ is equivalent to $\neg \exists x \text{ Likes}(x, \text{Parsnips})$.

We can go one step further: "Everyone likes ice cream" means that there is no one who does not like ice cream:

$\forall x \text{ Likes}(x, \text{IceCream})$ is equivalent to $\neg \exists x \neg \text{Likes}(x, \text{IceCream})$.

Because $\forall$ is really a conjunction over the universe of objects and $\exists$ is a disjunction, it should not be surprising that they obey De Morgan's rules

Thus, we do not really need both $\forall$ and $\exists$, just as we do not really need both $\wedge$ and $\vee$. Still, readability is more important, so we will keep both quantifiers.

03*First-Order Logic-Equality:

We can use the equality symbol to signify that two terms refer to the same object.

For example, Father (John) = Henry says that the object referred to by Father (John) and the object referred to by Henry are the same.

Because an interpretation fixes the referent of any term, determining the truth of an equality sentence is simply a matter of seeing that the referents of the two terms are the same object.

The equality can also be used with negation to insist that two terms are not the same object.

To say that Richard has at least two brothers, we would write $\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg(x=y)$.

The sentence $\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard})$ does not have the intended meaning.

It is true in the sample model, where Richard has only one brother. To see this, consider the extended interpretation in which both x and y are assigned to King John. The addition of $\neg(x=y)$ rules out such models.

04*The kinship domain:

Knowledge Representation (Knowledge Base): The set of information we know about our abstracted model.

Sentences are added to a knowledge base using TELL, such sentences are called assertions.

For example, we can assert that John is a king, Richard is a person, and all kings are persons:

TELL(KB, King(John))

TELL(KB, Person(Richard))

TELL(KB, $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$)

We can ask questions of the knowledge base using ASK. For example: ASK(KB, King(John))

Questions asked with ASK are called queries or goals.

We can ask quantified queries, such as ASK(KB, $\exists x \text{ Person}(x)$), the result is true. If we want to know what value of x

makes the sentence true, we will need a different function, ASKVARS, which we call with: ASKVARS(KB, Person(x)) and which yields a stream of answers. In this case there will be two answers: $\{x/\text{John}\}$ and $\{x/\text{Richard}\}$.

Such an answer is called a substitution or binding list.

05*First-Order Logic:

This domain defines the relationships between persons.

Clearly, the objects in our domain are people.

We have two unary predicates, Male and Female.

Kinship relations, parenthood, brotherhood, marriage, and so on, are represented by binary predicates: Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt and Uncle.

We use functions for Mother and Father, because every person has exactly one of each of these (at least according to nature's design).

one's mother is one's female parent:

$$\forall m, c \text{ Mother}(c) = m \Leftrightarrow \text{Female}(m) \wedge \text{Parent}(m, c)$$

One's husband is one's male spouse:

$$\forall w, h \text{ Husband}(h, w) \Leftrightarrow \text{Male}(h) \wedge \text{Spouse}(h, w)$$

Male and female are disjoint categories:

$$\forall x \text{ Male}(x) \Leftrightarrow \neg \text{Female}(x)$$

Parent and child are inverse relations:

$$\forall p, c \text{ Parent}(p, c) \Leftrightarrow \text{Child}(c, p)$$

A grandparent is a parent of one's parent:

$$\forall g, c \text{ Grandparent}(g, c) \Leftrightarrow \exists p \text{ Parent}(g, p) \wedge \text{Parent}(p, c)$$

A sibling is another child of one's parents:

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow x \neq y \wedge \exists p \text{ Parent}(p, x) \wedge \text{Parent}(p, y)$$

06*The natural numbers:

We describe here the theory of natural numbers or non-negative integers.

We need a predicate NatNum that will be true of natural numbers.

We need one constant symbol 0.

We need one function symbol S (successor). $S(0) = 1, S(1) = 2 \dots$

Natural numbers are defined recursively:

$$\text{NatNum}(0)$$

$$\forall n \text{ NatNum}(n) \Rightarrow \text{NatNum}(S(n))$$

That is, 0 is a natural number, and for every object n, if n is a natural number, then S(n) is a natural number.

So, the natural numbers are 0, S(0), S(S(0)), and so on

We also need axioms to constrain the successor function:

$$\forall n \ 0 \neq S(n)$$

$$\forall m, n \ m = n \Rightarrow S(m) = S(n)$$

Now we can define addition in terms of the successor function:

$$\forall m \ \text{NatNum}(m) \Rightarrow +(0, m) = m$$

$$\forall m, n \ \text{NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow +(S(m), n) = S(+(m, n))$$

The first of these axioms says that adding 0 to any natural number m gives m itself.

The second axiom reduces addition to repeated application of the successor function.

Simple Example: $+(2, 4) = +(S(1), 4) = S(+(1, 4)) = S(+(S(0), 4)) = S(S(+(0, 4))) = S(S(4)) = S(5) = 6$

07*Sets:

Constants, Predicates and Functions:

The empty set is a constant written as $\{\}$

There is one unary predicate, Set, which is true of sets

The binary predicate $x \in s$ (x is a member of set s)

The binary predicate $s1 \subseteq s2$ (set $s1$ is a subset of set $s2$)

The binary function are $s1 \cap s2$ (the intersection of two sets)

The binary function $s1 \cup s2$ (the union of two sets)

The binary function $\{x|s\}$ (the set resulting from adjoining element x to set s)

Axioms:

The only sets are the empty set and those made by adjoining something to a set:

$$\forall s \text{ Set}(s) \Leftrightarrow (s = \{\}) \vee (\exists x, s2 \text{ Set}(s2) \wedge s = \{x|s2\})$$

The empty set has no elements adjoined into it. In other words, there is no way to decompose $\{\}$ into a smaller set

and an element: $\neg \exists x, s \{x|s\} = \{\}$

Adjoining an element already in the set has no effect:

$$\forall x, s \quad x \in s \Leftrightarrow s = \{x|s\}$$

The only members of a set are the elements that were adjoined into it. We express this recursively, saying that x is a member of s if and only if s is equal to some set $s2$ adjoined with some element y , where either y is the same as x or x is a member of $s2$:

$$\forall x, s \quad x \in s \Leftrightarrow \exists y, s2 (s = \{y|s2\} \wedge (x = y \vee x \in s2))$$

A set is a subset of another set if and only if all the first set's members are members of the second set:

$$\forall s1, s2 \quad s1 \subseteq s2 \Leftrightarrow (\forall x \quad x \in s1 \Rightarrow x \in s2)$$

Two sets are equal if and only if each is a subset of the other:

$$\forall s1, s2 \quad (s1 = s2) \Leftrightarrow (s1 \subseteq s2 \wedge s2 \subseteq s1)$$

An object is in the intersection of two sets if and only if it is a member of both sets:

$$\forall x, s_1, s_2 \ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$$

An object is in the union of two sets if and only if it is a member of either set:

$$\forall x, s_1, s_2 \ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$$

08*Lists:

We can use the vocabulary of Lisp for lists:

Nil is the constant list with no elements

Cons (Construct), Append, First and Rest are functions

-Find is the predicate that does for lists what Member does for sets.

-List? is a predicate that is true only of lists.

-As with sets, it is common to use syntactic sugar in logical sentences involving lists.

The empty list is [].

-The term Cons(x, y), where y is a nonempty list, is written [x|y].

-The term Cons(x, Nil) (i.e., the list containing the element x) is written as [x].

-A list of several elements, such as [A,B,C], corresponds to the nested term Cons(A, Cons(B, Cons(C, Nil))).

Chapter 06 : Inference in First-Order Logic

01*Propositional vs First-Order Interface:

-Inference rules for quantifiers - Universal instantiation:

Consider the following sentence:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

Then it seems quite permissible to infer any of the following sentences:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

The rule of Universal Instantiation (UI for short) says that we can infer any sentence obtained by substituting a ground

term (a term without variables) for the variable. To write out the inference rule formally, we use the notion of substitutions, Let $\text{SUBST}(\theta, \alpha)$ denote the result of applying the substitution θ to the sentence α . Then the rule is written:

for any variable v and ground term g . For example, the three sentences given earlier are obtained with the substitutions $\{x/\text{John}\}$, $\{x/\text{Richard}\}$, and $\{x/\text{Father}(\text{John})\}$

-Inference rules for quantifiers - Existential Instantiation ($\exists$):

In the rule for Existential Instantiation, the variable is replaced by a single new constant symbol. The formal statement is as follows: for any sentence α , variable v , and constant symbol k that does not appear elsewhere in the knowledge base :

For example, from the sentence

$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$

we can infer the sentence

$\text{Crown}(C1) \wedge \text{OnHead}(C1, \text{John})$

as long as $C1$ does not appear elsewhere in the knowledge base. Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object. Of course, that name must not already belong to another object. In logic, the new name is called a Skolem constant ($d(x y)/dy = x y$ but not $\top$)

-Reduction to propositional inference:

Once we have rules for inferring non-quantified sentences from quantified sentences, it becomes possible to reduce first-order inference to propositional inference.

The first idea is that, just as an existentially quantified sentence can be replaced by one instantiation, a universally quantified sentence can be replaced by the set of all possible instantiations.

For example, suppose our knowledge base contains just the sentences

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

Then we apply UI to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge

base—in this case, $\{x/\text{John}\}$ and $\{x/\text{Richard}\}$. We obtain

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

and we discard the universally quantified sentence. Now, the knowledge base is essentially propositional. This concept is called **propositionalization**

-Entailed sentences:

In First-Order Logic (FOL), a sentence (or formula) α is said to be entailed by a knowledge base

(KB) if and only if α is true in every model (interpretation) in which the KB is true. This relationship is denoted by $\text{KB} \models \alpha$, which reads as "KB entails α ".

We have sketched an approach to first-order inference via propositionalization that is complete, that is, any entailed sentence can be proved.

The question of entailment for first-order logic is semi-decidable, that is, algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every non-entailed

sentence. In other words, if KB entails α then we can find a proof, but in case KB does not entail α , we cannot provide a proof. Since Our proof procedure can go on and on, generating more and more deeply nested terms, but we will not know whether it is stuck in a hopeless loop or whether the proof is just about to pop out.

02*Unification and lifting:

-Lifting:

Suppose we have the following sentences:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John}) \wedge \forall y \text{ Greedy}(y)$

How to proof Evil(John)?

if there is some substitution θ that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ .

This inference process can be captured as a single inference rule that we call Generalized Modus Ponens: For atomic sentences p_i , p_i' , and q , where there is a substitution θ such that $\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i)$, for all i :

There are $n+1$ premises to this rule: the n atomic sentences p_i' and the one implication. The conclusion is the result of applying the substitution θ to the consequent q . For our example:

p_1' is $\text{King}(\text{John})$

p_1 is $\text{King}(x)$

p_2' is $\text{Greedy}(\text{John})$

p_2 is $\text{Greedy}(x)$

θ is $\{x/\text{John}, y/\text{John}\}$

q is $\text{Evil}(x)$

$\text{SUBST}(\theta, q)$ is $\text{Evil}(\text{John})$.

Generalized Modus Ponens is a lifted version of Modus Ponens, it raises Modus Ponens from ground (variable-free) propositional logic to first-order logic.

-Unification:

Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called unification and is a key component of all first-order inference algorithms. The UNIFY algorithm takes two sentences and returns a unifier for them if one exists:

$\text{UNIFY}(p, q) = \theta$ where $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$

Let us look at some examples of how UNIFY should behave. Suppose we have a query

$\text{AskVars}(\text{Knows}(\text{John}, x))$: whom does John know?

Answers to this query can be found by finding all sentences in the knowledge base that unify with $\text{Knows}(\text{John}, x)$.

Here are the results of unification with four different sentences that might be in the knowledge base:

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x/\text{Jane}\}$

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Bill})) = \{x/\text{Bill}, y/\text{John}\}$

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Mother}(y))) = \{y/\text{John}, x/\text{Mother}(\text{John})\}$

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Elizabeth})) = \text{fail}$.

The last unification fails because x cannot take on the values John and Elizabeth at the same time. Now, remember that

$\text{Knows}(x, \text{Elizabeth})$ means “Everyone knows Elizabeth,” so we should be able to infer that John knows Elizabeth.

The problem arises only because the two sentences happen to use the same variable name, x . The problem can be avoided by

standardizing apart one of the two sentences being unified, which means renaming its variables to avoid name clashes. For

example, we can rename x in $\text{Knows}(x, \text{Elizabeth})$ to $x17$ (a new variable name) without changing its meaning. Now the

unification will work: $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x17, \text{Elizabeth})) = \{x/\text{Elizabeth}, x17/\text{John}\}$

-Storage and Retrieval:

Underlying the TELL and ASK functions used to inform and interrogate a knowledge base are the more primitive STORE and FETCH functions.

STORE(s) stores a sentence s into the knowledge base and FETCH(q)

returns all unifiers such that the query q unifies with some sentence in the knowledge base.

The problem we used to illustrate unification, finding all facts that unify with $\text{Knows}(\text{John}, x)$ —is an instance of FETCHing.

The simplest way to implement STORE and FETCH is to keep all the facts in one long list and unify each query against every element of the list.

Such a process is inefficient, but it works, and it's all you need to understand the rest of the chapter.

03*First-Order Some definitions:

In First-Order Logic (FOL), a term is an expression that represents an object in the domain of discourse. Terms can be constants, variables,

or function applications. Terms are used to construct more complex logical formulas and are fundamental building blocks in FOL.

In First-Order Logic (FOL), a ground term is a term that does not contain any variables.

Ground terms are composed exclusively of constants and function symbols applied to other ground terms. They are fully specified and represent concrete objects or entities in the domain of discourse.

Literal: A literal is an atomic proposition (or variable) or its negation. For example, P and $\neg P$ are literals.

In logic, a clause is a disjunction of literals. It is a basic building block used in various forms of logical expressions, particularly in Conjunctive Normal Form (CNF) and Disjunctive Normal Form (DNF). Clauses are central to the resolution method in automated theorem proving.

Disjunction: A disjunction is a logical "or" operation. For example, $P \vee Q$ is a disjunction of the literals P and Q .

Conjunction: A conjunction is a logical "and" operation. For example, $P \wedge Q$ is a conjunction of the literals P and Q .

In the context of logic and automated reasoning, the term "sound" refers to the correctness of an inference method. Specifically, an inference method is considered sound if it only produces conclusions that are logically entailed by the premises. In other words, if a method is sound, it will not infer any false conclusions from true premises.

In the context of logic and automated reasoning, the term "complete" refers to the ability of an inference method to derive all possible conclusions that logically follow from the given premises (facts and rules). Specifically, an inference method is considered complete if it can find a solution or derive a conclusion whenever one exists, given enough time and resources.

04*Horn clauses:

A Horn clause is a special type of clause in first-order logic (FOL) that has at most one positive literal. Horn clauses are particularly significant in logic programming and automated reasoning because they have desirable computational properties and are used in systems like Prolog.

Structure of Horn Clauses:

In first-order logic, a clause is typically a disjunction of literals (a literal is either an atomic formula or its negation). A

Horn clause can be defined as follows:

Definite Clause:

A Horn clause with exactly one positive literal. It has the form:

$$L_1 \vee \neg L_2 \vee \neg L_3 \vee \dots \vee \neg L_n$$

which can be written equivalently as

$$L_1 \leftarrow L_2 \wedge L_3 \wedge \dots \wedge L_n$$

Here, L_1 is the positive literal, and $L_2, L_3, \dots, L_n$ are negative literals (negated).

Fact:

A Horn clause with no negative literals. It is simply a single positive literal:
 L_1 which can be interpreted as $L_1 \leftarrow \text{true}$.

Goal Clause:

A Horn clause with no positive literals. It has the form:

$$\neg L_1 \vee \neg L_2 \vee \dots \vee \neg L_n$$

which can be written equivalently as:

$$\leftarrow L_1 \wedge L_2 \wedge \dots \wedge L_n$$

05*Forward chaining:

Every definite clause can be written as an implication whose premise is a conjunction of positive literals and whose conclusion is a single positive literal.

For example, the definite clause

$$(\neg L_1 \vee \neg L_2 \vee L_3)$$

can be written as the implication

$$(L_1 \wedge L_2) \Rightarrow L_3.$$

In the implication form, the sentence is easier to understand.

In Horn form, the premise is called the body and the conclusion is called the head.

The forward-chaining algorithm determines if a single proposition symbol q , the query, is entailed by a knowledge base of definite clauses.

It begins from known facts (positive literals) in the knowledge base.

If all the premises of an implication are known, then its conclusion is added to the set of known facts.

For example, if $L_{1,1}$ and Breeze are known and $(L_{1,1} \wedge \text{Breeze}) \Rightarrow B_{1,1}$ is in the knowledge base, then $B_{1,1}$ can be added.

This process continues until the query q is added or until no further inferences can be made.

It is easy to see that forward chaining is sound: every inference is essentially an application of Modus Ponens.

Forward chaining is also complete: every entailed atomic sentence will be derived.

06*First-Order definite clauses:

First-order definite clauses closely resemble propositional definite clauses: they are disjunctions of literals of which exactly, one is positive.

A definite clause either is atomic or is an implication whose antecedent is a conjunction of positive

literals and whose consequent is a single positive literal.

The following are first-order definite clauses:

$\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(y)$.

Unlike propositional literals, first-order literals can include variables, in which case those variables are assumed to be universally quantified. (Typically, we omit universal quantifiers when writing definite clauses.) Not every knowledge base can be converted into a set of definite clauses because of the single-positive-literal restriction, but many can.

07*First-Order Forward chaining:

The first forward-chaining algorithm we consider is a simple one.

Starting from the known facts, it triggers all the rules whose premises are satisfied, adding their conclusions to the known facts.

The process repeats until the query is answered (assuming that just one answer is required) or no new facts are added.

First, it is sound, because every inference is just an application of Generalized Modus Ponens, which is sound.

Second, it is complete for definite clause knowledge bases, that is, it answers every query whose answers are entailed by any knowledge base of definite clauses.

08*First-Order Logic conjunctive normal form(CNF):

A first-order logic sentence is in **CNF** if it is a conjunction of clauses where each clause is a disjunction of literals. Literals can contain variables, which are assumed to be universally quantified. For example, the sentence:

$\forall x \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Becomes in CNF:

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$

Every sentence of first-order logic can be converted into an inferentially equivalent CNF sentence.

In particular, the CNF sentence will be unsatisfiable just when the original sentence is unsatisfiable.

This leads to a basis for doing proofs by contradiction on the CNF sentences.

09*First-Order Logic conversion to CNF:

1. Eliminate Implications and Biconditionals:

Replace implications $P \Rightarrow Q$ with $\neg P \vee Q$.

Replace biconditionals $P \Leftrightarrow Q$ with $(\neg P \vee Q) \wedge (\neg Q \vee P)$.

2. Move Negations Inward (Negation Normal Form):

Use De Morgan's laws to move negations inward, eliminating double negations:

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

3. Standardize Variables:

Ensure that each quantifier uses a unique variable to avoid confusion during Skolemization.

4. Skolemization:

Eliminate existential quantifiers by introducing Skolem constants (for ground terms) or Skolem functions (for terms with variables):

If no universal quantifier precedes the existential quantifier, replace the variable with a constant symbol.

$$\exists y P(x, y) \rightarrow P(x, A)$$

Where A is a Skolem Constant

If a universal quantifier precedes the existential quantifier, replace the variable with a function of the universal quantifier variable:

$$\forall x \exists y P(x, y) \rightarrow \forall x P(x, f(x))$$

The Skolem function $f(x)$ is used to replace y in the context of the universally quantified variable x .

5.Drop Universal Quantifiers:

Once all existential quantifiers have been eliminated, drop the universal quantifiers for the remaining variables, assuming they are universally quantified.

6.Distribute $\wedge$ over $\vee$ (Distributive Laws):

Apply the distributive laws to move the formula into CNF:

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$$

$$(P \vee Q) \wedge R \equiv (P \wedge R) \vee (Q \wedge R)$$

10*Resolution Inference rule:

Resolution:

is a Method using clauses to derive new clauses and prove satisfiability or un-satisfiability. Propositional literals are complementary if one is the negation of the other. First-order literals are complementary if one unifies with the negation of the other.

Resolution Inference Rule:

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$.

For example, we can resolve the two clauses

$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)]$ and $[\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v)]$

by eliminating the complementary literals $\text{Loves}(G(x), x)$ and $\neg \text{Loves}(u, v)$, with unifier $\theta = \{u/G(x), v/x\}$, to produce the resolvent clause

$[\text{Animal}(F(x)) \vee \neg \text{Kills}(G(x), x)]$

11*Resolution Refutation (Proof by contradiction):

Resolution refutation proves that $\text{KB} \models \alpha$ by proving $\text{KB} \wedge \neg \alpha$ unsatisfiable, that is, by deriving the empty clause.

Main resolution refutation steps:

1.Convert KB and $\neg \alpha$ to CNF with ground terms and universal variable only.

2.Apply repeatedly the resolution rule.

3.Stop when empty set (contradiction) is derived or no more resolvents (conclusions) follow.

If we derive a contradiction, then the conclusion follow from the axioms.

If we can't apply anymore resolvents, then the conclusion cannot be proved from the axioms.

Resolution Refutation is sound and complete.

Example:

The sentences in CNF are:

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$

$\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono})$

$\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x)$

$\neg \text{Missile}(x) \vee \text{Weapon}(x)$

$\text{Owns}(\text{Nono}, \text{M1})$

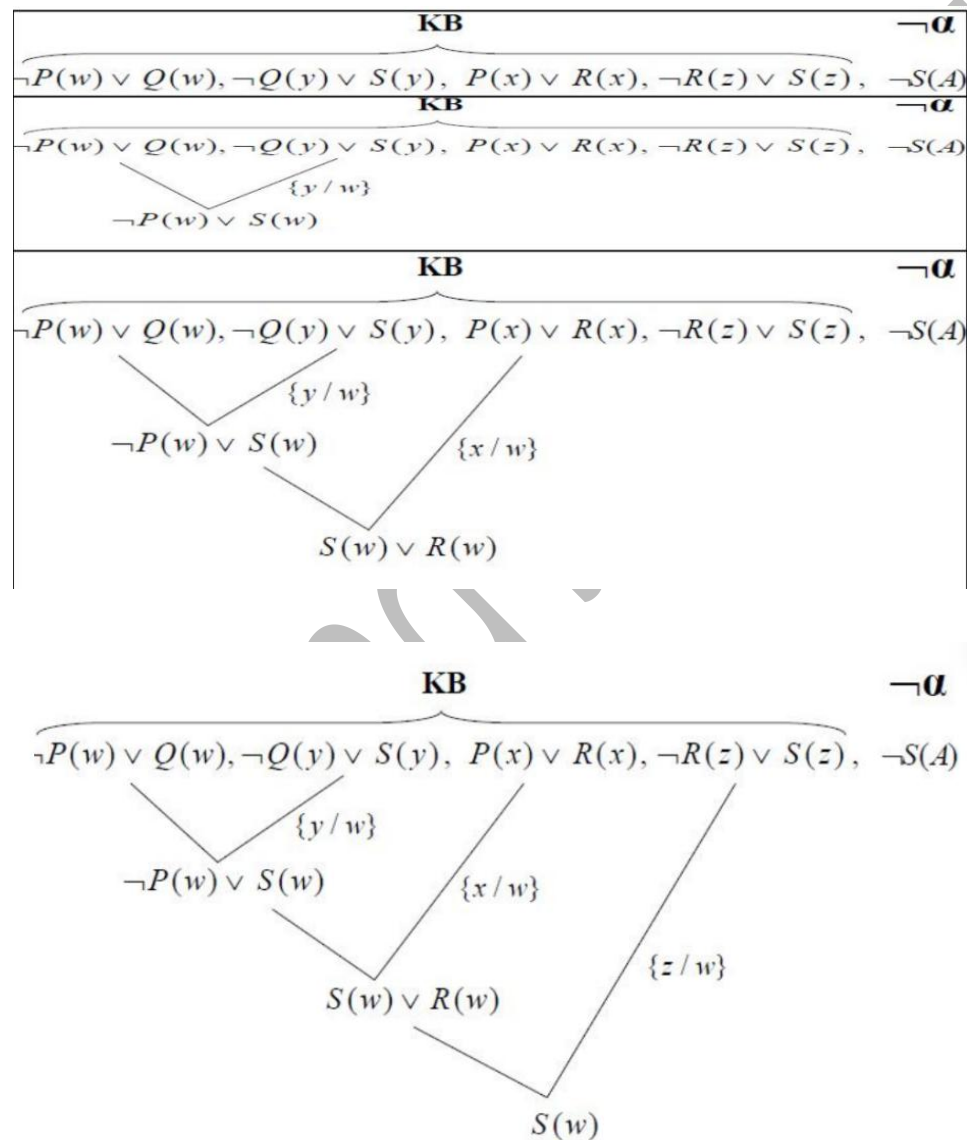
$\text{Missile}(\text{M1})$

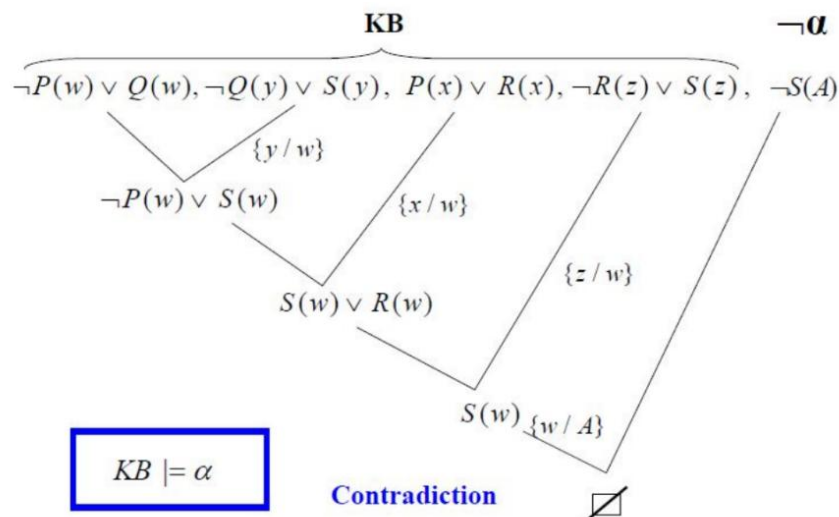
$\text{American}(\text{West})$

$\text{Enemy}(\text{Nono}, \text{America})$.

We also include the negated goal

$\neg \text{Criminal}(\text{West})$.





Discrete Mathematics:

Chapter 01: The Foundations – Logic and Proofs[1]

01*Positional (Propositional) Logic:

What is a Proposition?

A proposition is a statement that has a definite truth value. It is either true (T) or false (F), but not both.

Examples:

- 7 is greater than 5. → True
- Paris is the capital of Italy. → False

Questions, commands, and open sentences (such as $x > 3$) are not propositions because they do not have a fixed truth value.

Compound Propositions:

A compound proposition is formed by combining one or more propositions using logical operators.

1.Negation ($\neg p$):

Negation changes the truth value of a proposition.

- If p is true, then $\neg p$ is false.
- If p is false, then $\neg p$ is true.

Example:

- p : "It is raining."
- $\neg p$: "It is not raining."

2.Conjunction ($p \wedge q$):

A conjunction means " p and q ."

It is true only when both propositions are true.

Example:

- p : "I finished my homework."
- q : "I submitted the assignment."
- $p \wedge q$: "I finished my homework and submitted the assignment."

3.Disjunction ($p \vee q$):

A disjunction means " p or q ."

It is false only when both propositions are false.

Example:

- p : "I will study."
- q : "I will exercise."
- $p \vee q$ means at least one of them is true.

4. Conditional Statement ($p \rightarrow q$):

A conditional statement means "If p, then q."

It is false only when p is true and q is false.

Example:

- If it rains, then the ground becomes wet.

5. Biconditional Statement ($p \leftrightarrow q$):

A biconditional means "p if and only if q."

It is true when both propositions have the same truth value (both true or both false).

Example:

- A figure is a square if and only if it has four equal sides and four right angles.

Truth Table:

A truth table shows the truth value of a compound proposition for every possible combination of truth values.

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Truth tables are useful for checking logical relationships and verifying arguments.

Converse, Inverse, and Contrapositive:

Given the conditional statement:

$p \rightarrow q$ (If p, then q)

Three related statements can be formed.

1. Converse:

Swap the hypothesis and conclusion.

$q \rightarrow p$

Example:

- Original: If it rains, then the road is wet.

- Converse: If the road is wet, then it is raining.

The converse is not logically equivalent to the original statement.

2. Inverse:

Negate both parts of the original statement.

$\neg p \rightarrow \neg q$

Example:

- If it does not rain, then the road is not wet.

The inverse is also not logically equivalent to the original statement.

3. Contrapositive:

Swap the hypothesis and conclusion, then negate both.

$\neg q \rightarrow \neg p$

Example:

- If the road is not wet, then it did not rain.

The contrapositive is logically equivalent to the original conditional statement. This means they always have the same truth value.

Chapter 02 : The Foundations – Logic and Proofs[2]:

01*Propositional Equivalence:

Tautology, Contradiction, and Contingency

Logical statements can be classified based on their truth values.

1.Tautology:

A tautology is a proposition that is always true, no matter what truth values are assigned to its variables.

Example:

" $p \vee \neg p$ "

This expression is always true because either p is true or p is false.

2.Contradiction:

A contradiction is a proposition that is always false, regardless of the truth values of its variables.

Example:

" $p \wedge \neg p$ "

A proposition cannot be both true and false at the same time, so this expression is always false.

3.Contingency:

A contingency is a proposition that is true in some cases and false in others. Its truth value depends on the values of its variables.

Example:

" $p \rightarrow q$ "

Since its truth value changes with different inputs, it is neither a tautology nor a contradiction.

Logical Equivalence:

Two propositions are logically equivalent if they always have the same truth value for every possible combination of variables.

The symbol for logical equivalence is $\equiv$.

Example:

$$\neg \neg p \equiv p$$

To verify logical equivalence, construct a truth table for both propositions. If the final columns are identical, the propositions are logically equivalent.

Common Logical Equivalence Laws:

Law	Equivalent Expression
Double Negation	$p \equiv \neg \neg p$
Identity Laws	$p \wedge T \equiv p$, $p \vee F \equiv p$
Domination Laws	$p \vee T \equiv T$, $p \wedge F \equiv F$
Idempotent Laws	$p \vee p \equiv p$, $p \wedge p \equiv p$
Negation Laws	$p \vee \neg p \equiv T$, $p \wedge \neg p \equiv F$
Commutative Laws	$p \vee q \equiv q \vee p$, $p \wedge q \equiv q \wedge p$
Associative Laws	$(p \vee q) \vee r \equiv p \vee (q \vee r)$, $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
Distributive Laws	$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$, $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
De Morgan's Laws	$\neg(p \wedge q) \equiv \neg p \vee \neg q$, $\neg(p \vee q) \equiv \neg p \wedge \neg q$

02*Predicates and Quantifiers:

Predicate:

A predicate is a statement that contains one or more variables. Unlike a proposition, a predicate does not have a truth value until the variables are assigned specific values.

A predicate is usually written as $P(x)$, where x is the variable.

Examples:

- " $P(x): x > 5$ "

- " $Q(x): x$ is an even number"

These statements become propositions only after replacing x with a specific value.

Example:

- " $P(8): 8 > 5 \rightarrow \text{True}$ "

- " $P(3): 3 > 5 \rightarrow \text{False}$ "

Quantifiers:

A quantifier specifies how many elements in a given set satisfy a predicate. The two main quantifiers are the universal quantifier and the existential quantifier.

1. Universal Quantifier (For All):

The universal quantifier is written as $\forall x$ and means "for every element in the domain."

Notation:

- " $\forall x P(x)$ "

This statement is true only if $P(x)$ is true for every value of x in the domain.

Example:

- " $\forall x (x + 0 = x)$ "

This statement is true because adding zero does not change any number.

2. Existential Quantifier (There Exists):

The existential quantifier is written as $\exists x$ and means "there is at least one element in the domain."

Notation:

- " $\exists x P(x)$ "

This statement is true if at least one value of x satisfies the predicate.

Example:

- " $\exists x (x^2 = 16)$ "

This statement is true because 4 and -4 satisfy the equation.

Logical Equivalences Involving Quantifiers:

Negating a quantified statement changes both the quantifier and the predicate.

Negation of the Universal Quantifier

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

Meaning:

- "It is not true that every element satisfies P" is equivalent to "There exists at least one element that does not satisfy P."

Negation of the Existential Quantifier:

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

Meaning:

- "There does not exist an element that satisfies P" is equivalent to "Every element does not satisfy P."

These equivalence rules are important when simplifying logical expressions and proving mathematical statements.

03*Nested Quantifiers:

Nested quantifiers occur when a statement contains more than one quantifier. The order of the quantifiers is important because changing the order may change the meaning of the statement.

Examples:

$$\neg(\forall x \exists y P(x, y))$$

$$\neg(\exists s x \forall y P(x, y))$$

These two statements are generally not equivalent.

Example:

$$\forall x \exists y (x + y = 0)$$

Meaning:

For every value of x, there is a value of y such that their sum is zero.

This statement is true because for any number x, we can choose $y = -x$.

Another example:

$$\exists x \forall y (x + y = y)$$

Meaning: There exists a number x that, when added to any number y, does not change its value.

This statement is true because $x = 0$ satisfies the condition.

04*Mathematical Statements Involving Quantifiers:

Quantifiers are commonly used to write mathematical definitions and theorems in a precise way.

Examples:

- " $\forall x (x + 0 = x)$ "

Meaning: Every number remains the same when zero is added.

- " $\exists x (x^2 = 25)$ "

Meaning: There is at least one number whose square is 25.

- " $\forall x (x^2 \geq 0)$ "

Meaning: The square of every real number is non-negative.

These statements clearly describe mathematical properties using quantifiers.

05*Mathematical Statements Involving Nested Quantifiers:

Nested quantifiers are often used to describe relationships between two or more variables.

Example 1:

" $\forall x \exists y (y > x)$ "

Meaning: For every number x , there exists another number y that is greater than x .

This statement is true for the set of real numbers.

Example 2:

" $\exists x \forall y (x \geq y)$ "

Meaning: There exists one number that is greater than or equal to every number in the domain.

This statement is false for the set of real numbers because there is no largest real number.

Note:

The order of quantifiers matters.

For example:

- " $\forall x \exists y P(x, y)$ " means each value of x may have a different value of y .

- " $\exists y \forall x P(x, y)$ " means one fixed value of y must work for every value of x .

Although the statements use the same quantifiers, they usually have different meanings and truth values.